

การทดลองที่ 10 วงจรจำลองการทำงานของ CPU อย่างง่ายโดยใช้วิธี Schematic บนบอร์ด FPGA EDGE Spartan-7

วัตถุประสงค์

1. เพื่อเสริมสร้างความเข้าใจของนักศึกษาเกี่ยวกับกลไกการประมวลผลของ CPU
2. เพื่อให้นักศึกษาฝึกการใช้งาน FPGA
3. เพื่อให้นักศึกษาเห็นความเชื่อมโยงระหว่างวงจรดิจิทัลที่ออกแบบเอง กับการทำงานจริงของ CPU โดยสร้างส่วน ALU ACC และ FLAG เป็นวงจรจริงใน FPGA และทำหน้าที่ของ IR PC และ CU ด้วยตนเอง
4. เพื่อให้นักศึกษาฝึกเขียนผังวงจรแบบ Schematic จากบล็อกย่อย และกำหนดขา (Pin Assignment) ให้ตรงกับอุปกรณ์จริงบนบอร์ด

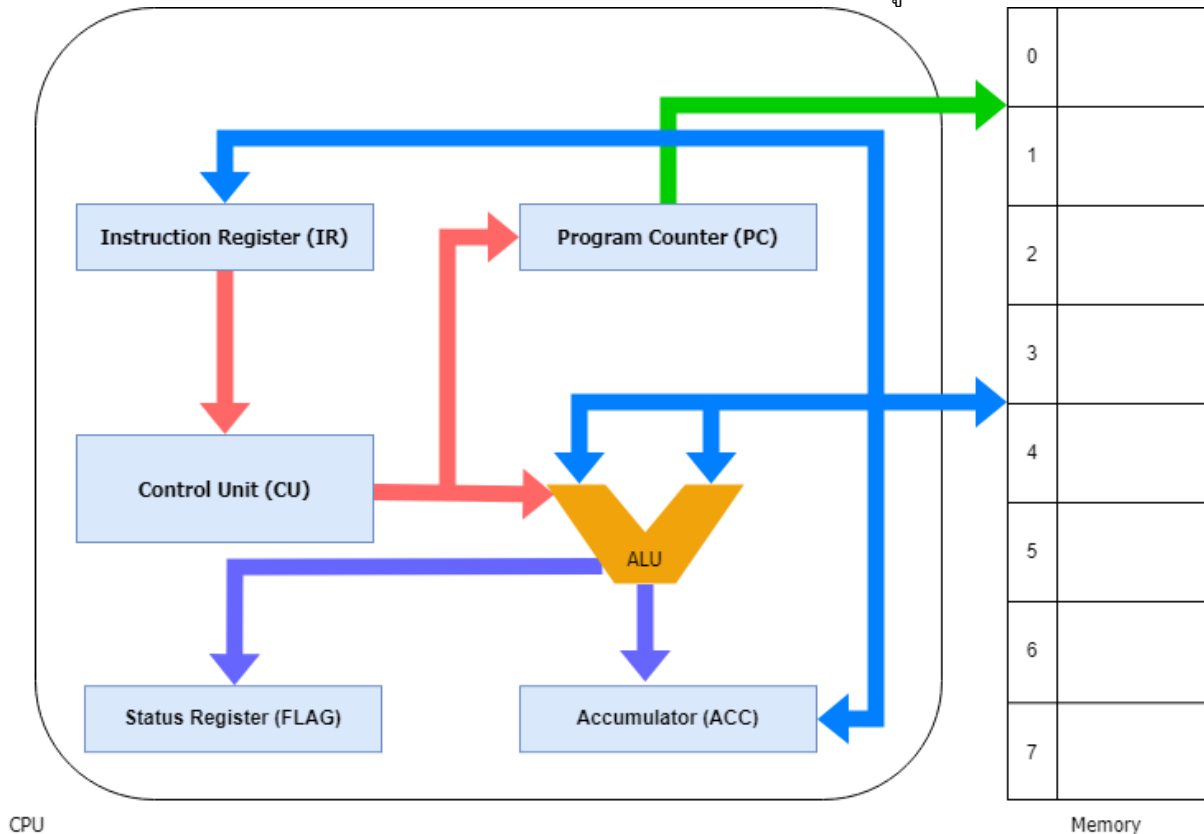
**** หมายเหตุ ให้อ่านเอกสารการทดลองและเอกสารประกอบให้ครบก่อนเริ่มทำการทดลอง ****

บทนำ

หน่วยประมวลผลกลาง (Central Processing Unit: CPU) เป็นองค์ประกอบสำคัญของระบบคอมพิวเตอร์ ทำหน้าที่ประมวลผลข้อมูลตามชุดคำสั่งที่กำหนด

1. องค์ประกอบหลักของ CPU

ประกอบด้วย 6 ส่วนหลักที่ทำงานประสานกันเพื่อประมวลผลคำสั่งและข้อมูล



รูปที่ 1 องค์ประกอบหลักของ CPU และการทำงานร่วมกับหน่วยความจำ

1.1. รีจิสเตอร์คำสั่ง (Instruction Register: IR)

- 1.1.1. ทำหน้าที่จัดเก็บคำสั่งที่กำลังประมวลผลในขณะนั้น
- 1.1.2. เป็นจุดเชื่อมต่อระหว่างหน่วยความจำและวงจรการประมวลผลของ CPU

1.2. ตัวนับโปรแกรม (Program Counter: PC)

- 1.2.1. บันทึกที่อยู่ของคำสั่งถัดไปที่จะถูกเรียกมาประมวลผล
- 1.2.2. มีการปรับค่าโดยอัตโนมัติหลังจากการเรียกคำสั่งแต่ละครั้ง หรือถูกปรับโดยคำสั่งกระโดด (Jump) และการเรียกโปรแกรมน้อย (Subroutine)

1.3. หน่วยควบคุม (Control Unit: CU)

- 1.3.1. ทำหน้าที่ควบคุมการทำงานของส่วนประกอบอื่น ๆ ใน CPU
- 1.3.2. ถอดรหัสคำสั่งจาก IR และส่งสัญญาณควบคุมไปยังส่วนต่าง ๆ ของ CPU
- 1.3.3. ประสานงานระหว่างหน่วยความจำและหน่วยคำนวณและตรรกะ

1.4. รีจิสเตอร์สถานะ (Status Register หรือ FLAG)

- 1.4.1. บันทึกสถานะของการประมวลผลล่าสุด เช่น ผลลัพธ์เป็นศูนย์, มีการทด, มีเครื่องหมายลบ
- 1.4.2. ใช้ในการตัดสินใจสำหรับคำสั่งแบบมีเงื่อนไข และการจัดการกับข้อยกเว้น (Exceptions)

1.5. ตัวสะสม (Accumulator: ACC)

- 1.5.1. เป็นรีจิสเตอร์พิเศษที่ใช้เก็บผลลัพธ์ชั่วคราวจากการคำนวณของหน่วยคำนวณและตรรกะ
- 1.5.2. มักเป็นตัวตั้งหรือตัวรับในการดำเนินการทางคณิตศาสตร์หรือตรรกะ

1.6. หน่วยคำนวณและตรรกะ (Arithmetic and Logic Unit: ALU)

- 1.6.1. ทำการคำนวณทางคณิตศาสตร์ (บวก, ลบ, คูณ, หาร)
- 1.6.2. ดำเนินการทางตรรกะ (AND, OR, NOT, XOR)
- 1.6.3. ส่งผลลัพธ์ไปยัง ACC และปรับค่า FLAG ตามผลการคำนวณ

2. การทำงานของ CPU ร่วมกับหน่วยความจำ (Memory)

การทำงานของ CPU ร่วมกับ Memory เป็นกระบวนการต่อเนื่องที่เรียกว่า วงจรการเรียกและปฏิบัติตามคำสั่ง (Fetch-Execute Cycle) ประกอบด้วยขั้นตอนดังนี้

- 2.1. การเรียกคำสั่ง (Fetch) CPU ใช้ค่าใน PC เพื่อระบุตำแหน่งของคำสั่งใน Memory และเรียกคำสั่งมาเก็บใน IR
- 2.2. การถอดรหัสคำสั่ง (Decode) CU วิเคราะห์คำสั่งใน IR เพื่อกำหนดการดำเนินการที่จำเป็น
- 2.3. การเรียกข้อมูล (Fetch Operands) หากคำสั่งต้องการข้อมูลเพิ่มเติม CPU จะเรียกข้อมูลจาก Memory หรือรีจิสเตอร์อื่น ๆ
- 2.4. การปฏิบัติตามคำสั่ง (Execute) ALU ดำเนินการตามคำสั่ง โดยใช้ข้อมูลจาก ACC หรือรีจิสเตอร์อื่น ๆ
- 2.5. การจัดเก็บผลลัพธ์ (Store Result) ผลลัพธ์จะถูกเขียนกลับไปยัง Memory หรือรีจิสเตอร์ตามที่ระบุในคำสั่ง

3. การสื่อสารระหว่าง CPU และ RAM ดำเนินการผ่านบัสระบบ (System Bus) ซึ่งประกอบด้วย

- 3.1. บัสข้อมูล (Data Bus) สำหรับการส่งข้อมูลระหว่าง CPU และ RAM
- 3.2. บัสที่อยู่ (Address Bus) ใช้ระบุตำแหน่งของข้อมูลใน RAM
- 3.3. บัสควบคุม (Control Bus) สำหรับส่งสัญญาณควบคุมระหว่าง CPU และอุปกรณ์อื่น ๆ

4. คำสั่ง (Instructions) และการประมวลผล

คำสั่งคอมพิวเตอร์เป็นชุดของรหัสที่บอก CPU ว่าต้องดำเนินการอย่างไร โดยทั่วไปคำสั่งประกอบด้วยสองส่วนหลัก

- 4.1. รหัสปฏิบัติการ (Operation Code หรือ Opcode) ระบุการดำเนินการที่ต้องทำ เช่น บวก, ลบ, กระโดด
- 4.2. ตัวถูกดำเนินการ (Operand) ข้อมูลหรือตำแหน่งของข้อมูลที่จะใช้ในการดำเนินการ
คำสั่งถูกจัดเก็บใน Memory และถูกเรียกมาประมวลผลทีละคำสั่งตามลำดับที่กำหนดโดย PC เว้นแต่จะมีคำสั่งกระโดดหรือเรียกโปรแกรมย่อย
- 4.3. ประเภทของคำสั่งที่พบบ่อยในการทำงานของ CPU ได้แก่
 - 4.3.1. คำสั่งการโอนข้อมูล (Data Transfer Instructions)
 - 4.3.2. คำสั่งทางคณิตศาสตร์และตรรกะ (Arithmetic and Logical Instructions)
 - 4.3.3. คำสั่งควบคุมการทำงาน (Control Flow Instructions)
 - 4.3.4. คำสั่งการจัดการหน่วยความจำ (Memory Management Instructions)
 - 4.3.5. คำสั่งการป้อนข้อมูลและแสดงผล (Input/Output Instructions)

หลักการเชื่อมโยงกับการทำงานของ CPU

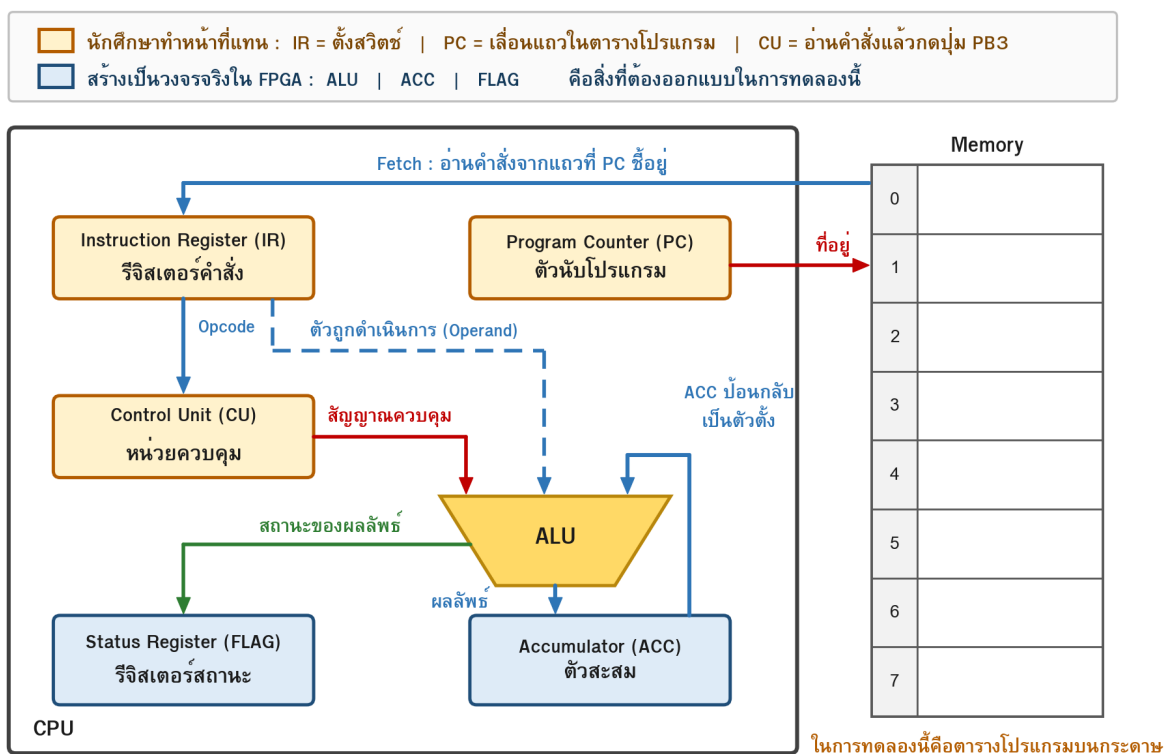
1. แบ่ง CPU ออกเป็นสองส่วน แล้วสร้างทีละส่วน

เมื่อมองการทำงานของ CPU ตามหัวข้อหน้าข้างต้น จะเห็นว่าองค์ประกอบทั้ง 6 ส่วน แบ่งได้เป็นสองกลุ่มตามหน้าที่ที่ต่างกันอย่างชัดเจน

- 1.1. ส่วนทางเดินข้อมูล (Datapath) ได้แก่ ALU ACC และ FLAG ทำหน้าที่คำนวณและจดจำ คือรับตัวเลขเข้ามา คำนวณ เก็บผลลัพธ์ และบอกสถานะของผลลัพธ์
- 1.2. ส่วนควบคุม (Control) ได้แก่ IR PC และ CU ทำหน้าที่สั่งการ คือบอกว่าจะคำนวณอะไร ด้วยข้อมูลใด และคำสั่งใดมาก่อนมาหลัง

การทดลองนี้ให้นักศึกษาสร้างเฉพาะส่วนทางเดินข้อมูลเป็นวงจรจริงใน FPGA ส่วนควบคุมนั้นให้นักศึกษารับบทบาททำหน้าที่แทนด้วยตนเอง โดยอ่านคำสั่งจากตารางโปรแกรม ตั้งค่าสวิตช์ แล้วกดปุ่มเพื่อสั่งให้วงจรทำงานทีละคำสั่ง

เหตุผลที่แบ่งเช่นนี้คือ ถ้าสร้างทั้ง 6 ส่วนพร้อมกันตั้งแต่แรก เมื่อวงจรทำงานผิดพลาด จะแยกไม่ออกว่าความผิดพลาดเกิดจากส่วนคำนวณหรือส่วนควบคุม การให้ส่วนควบคุมเป็นการทำงานด้วยมือของนักศึกษา ซึ่งตรวจสอบได้ทุกขั้นตอน ให้ความผิดพลาดที่พบทั้งหมดมาจากวงจรที่ออกแบบเองเท่านั้น และทำให้เห็นทางเดินของข้อมูลในแต่ละคำสั่งได้อย่างชัดเจน



รูปที่ 2 องค์ประกอบของ CPU กับส่วนที่สร้างเป็นวงจรจริงและส่วนที่นักศึกษาทำหน้าที่แทน

2. ตารางจับคู่องค์ประกอบของ CPU กับอุปกรณ์ในการทดลอง

ตารางที่ 1 สิ่งที่ทำหน้าที่แทนแต่ละองค์ประกอบของ CPU ในการทดลองนี้

องค์ประกอบของ CPU	หน้าที่ตามทฤษฎี	สิ่งที่ทำหน้าที่แทนในการทดลองนี้	ผู้ทำ
Memory	เก็บคำสั่งและข้อมูล	ตารางโปรแกรมบนกระดาน ที่อยู่ 0 ถึง 7	นักศึกษา
PC	ชี้ที่อยู่ของคำสั่งถัดไป	การเลื่อนไปยังแถวถัดไปของตารางโปรแกรม	นักศึกษา
IR	เก็บคำสั่งที่กำลังประมวลผล	ตำแหน่งของสวิตช์เลื่อน 11 ตัว (รหัสคำสั่ง 3 บิต + ค่าคงที่ 8 บิต)	นักศึกษา
CU	ถอดรหัสคำสั่งและออกสัญญาณควบคุม	นักศึกษาอ่านคำสั่ง ตั้งสวิตช์ แล้วกดปุ่ม PB3	นักศึกษา
สัญญาณนาฬิกา	กำหนดจังหวะของแต่ละคำสั่ง	กดปุ่ม PB3 หนึ่งครั้ง เท่ากับหนึ่งคำสั่ง	นักศึกษา
ALU	คำนวณตามรหัสคำสั่ง	วงจร ALU ที่ออกแบบเองใน FPGA	วงจร
ACC	เก็บผลลัพธ์และเป็นตัวตั้งของคำสั่งถัดไป	รีจิสเตอร์ 8 บิตใน FPGA แสดงผลที่ 7-segment หลักที่ 1 และ 0	วงจร
FLAG	บันทึกสถานะของผลลัพธ์ล่าสุด	รีจิสเตอร์ 3 บิตใน FPGA แสดงผลที่ LED D2 D3 และ D4	วงจร

3. รูปแบบของคำสั่งในการทดลองนี้

จากหัวข้อคำสั่ง (Instructions) และการประมวลผล ในบทนำ คำสั่งหนึ่งคำสั่งประกอบด้วยรหัสปฏิบัติการ (Opcode) และตัวถูกดำเนินการ (Operand) ในการทดลองนี้คำสั่งหนึ่งคำสั่งมีขนาด 11 บิต แบ่งเป็นรหัสปฏิบัติการ 3 บิต และตัวถูกดำเนินการ 8 บิต ดังนี้

$$\text{คำสั่ง 11 บิต} = [\text{OP2 OP1 OP0}] [\text{D7 D6 D5 D4 D3 D2 D1 D0}]$$

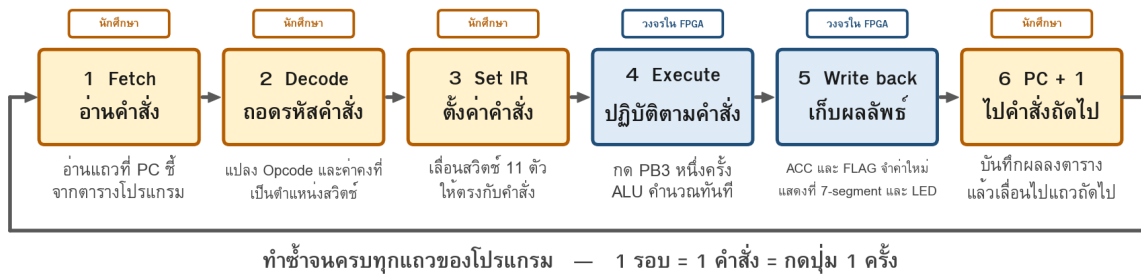
สังเกตว่าคำสั่งระบุตัวถูกดำเนินการมาเพียงตัวเดียว ส่วนตัวตั้งอีกตัวหนึ่งคือค่าใน ACC เสมอ และผลลัพธ์ก็ถูกเก็บกลับไป ACC เสมอ เขียนเป็นความสัมพันธ์ได้ว่า

$$\text{ACC} \leftarrow \text{ACC} \text{ (การดำเนินการตาม OP) ค่าคงที่}$$

สถาปัตยกรรมที่คำสั่งอ้างถึงตัวถูกดำเนินการเพียงตัวเดียวเช่นนี้เรียกว่า เครื่องแบบตัวสะสม (Accumulator Machine) หรือเครื่องแบบหนึ่งที่อยู่ (One-address Machine) ซึ่งเป็นรูปแบบของ CPU ยุคแรกและยังเป็นพื้นฐานของไมโครคอนโทรลเลอร์ขนาดเล็กจำนวนมากในปัจจุบัน ข้อดีคือคำสั่งสั้นและวงจรเรียบง่าย ข้อเสียคือต้องผ่าน ACC ทุกครั้ง

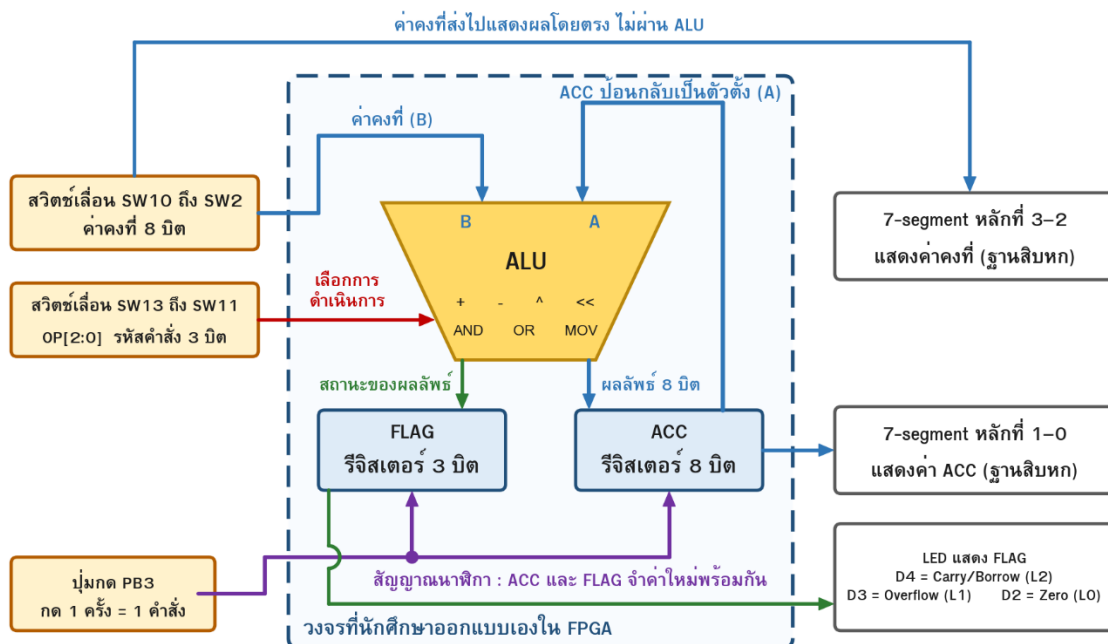
4. วงรอบการทำงานหนึ่งคำสั่งในการทดลองนี้

วงรอบการเรียกและปฏิบัติตามคำสั่ง (Fetch-Execute Cycle) ตามที่กล่าวในบทนำ จะเกิดขึ้นจริงในการทดลองนี้ทุกครั้งที่เกิดปุ่มหนึ่งครั้ง โดยแบ่งหน้าที่ระหว่างนักศึกษากับวงจรดังรูปที่ 3



รูปที่ 3 วงรอบการทำงานหนึ่งคำสั่ง และการแบ่งหน้าที่ระหว่างนักศึกษากับวงจร

ขั้นตอนที่ 1 ถึง 3 และขั้นตอนที่ 6 คือหน้าที่ของ PC IR และ CU ซึ่งนักศึกษาทำเองด้วยมือ ส่วนขั้นตอนที่ 4 และ 5 คือหน้าที่ของ ALU ACC และ FLAG ซึ่งวงจรใน FPGA ทำให้ทันทีที่เกิดปุ่ม การที่นักศึกษาต้องเดินที่ละขั้นตอนเช่นนี้ทำให้เห็นว่า CPU จริงทำงานอย่างไรในหนึ่งวงรอบสัญญาณนาฬิกา ต่างกันเพียงที่ CPU จริงทำครบทุกขั้นตอนเองด้วยความเร็วหลายล้านครั้งต่อวินาที



รูปที่ 4 ทางเดินข้อมูลของวงจรที่ต้องออกแบบ แสดงหน้าที่ของแต่ละส่วนและทิศทางของสัญญาณ

รูปที่ 4 แสดงหน้าที่และทิศทางการไหลของข้อมูลเท่านั้น ไม่ใช่ผังวงจรที่นำไปเขียนตามได้ทันที นักศึกษาต้องออกแบบวงจรภายในแต่ละบล็อกเอง และกำหนดเองว่าจะแบ่งบล็อกอย่างไร

6. FLAG

- 6.1. FLAG คือข้อมูลเพิ่มเติมเกี่ยวกับผลลัพธ์ ที่ตัวเลข 8 บิตใน ACC บอกไม่ได้ด้วยตัวเอง เช่น ผลลัพธ์เป็นศูนย์พอดีหรือไม่ และผลลัพธ์เกินขอบเขตที่ 8 บิตรองรับได้หรือไม่
- 6.2. ใน CPU จริง CU จะอ่านค่า FLAG เพื่อตัดสินใจว่าจะทำคำสั่งถัดไปหรือจะกระโดดไปที่อยู่อื่น ซึ่งก็คือคำสั่งแบบมีเงื่อนไขตามที่กล่าวไว้ในบทนำ
- 6.3. ในการทดลองนี้ นักศึกษาซึ่งทำหน้าที่ CU คือผู้ที่มีไฟ LED แล้วตัดสินใจเอง เช่นกำหนดว่าถ้าไฟ Zero ติด ให้ข้ามไปทำคำสั่งที่อยู่ที 6 การทำเช่นนี้คือการจำลองคำสั่งกระโดดแบบมีเงื่อนไขด้วยมือ
- 6.4. ให้ระวังความแตกต่างระหว่าง Carry กับ Overflow เพราะไม่ใช่สิ่งเดียวกัน Carry คือการทศออกจากบิตที่ 7 เมื่อตีความตัวเลขแบบไม่มีเครื่องหมาย ส่วน Overflow คือกรณีที่ผลลัพธ์ผิดพลาดเมื่อตีความตัวเลขแบบมีเครื่องหมาย ทั้งสองอาจติดพร้อมกัน ติดอย่างใดอย่างหนึ่ง หรือไม่ติดเลยก็ได้

7. การต่อยอดทำให้เป็น CPU

เมื่อทำการทดลองนี้สำเร็จ นักศึกษาจะได้ทางเดินข้อมูลของ CPU ที่ทำงานได้จริง สิ่งที่ต้องเพิ่มเพื่อให้กลายเป็น CPU ที่เดินโปรแกรมได้เองมีดังนี้

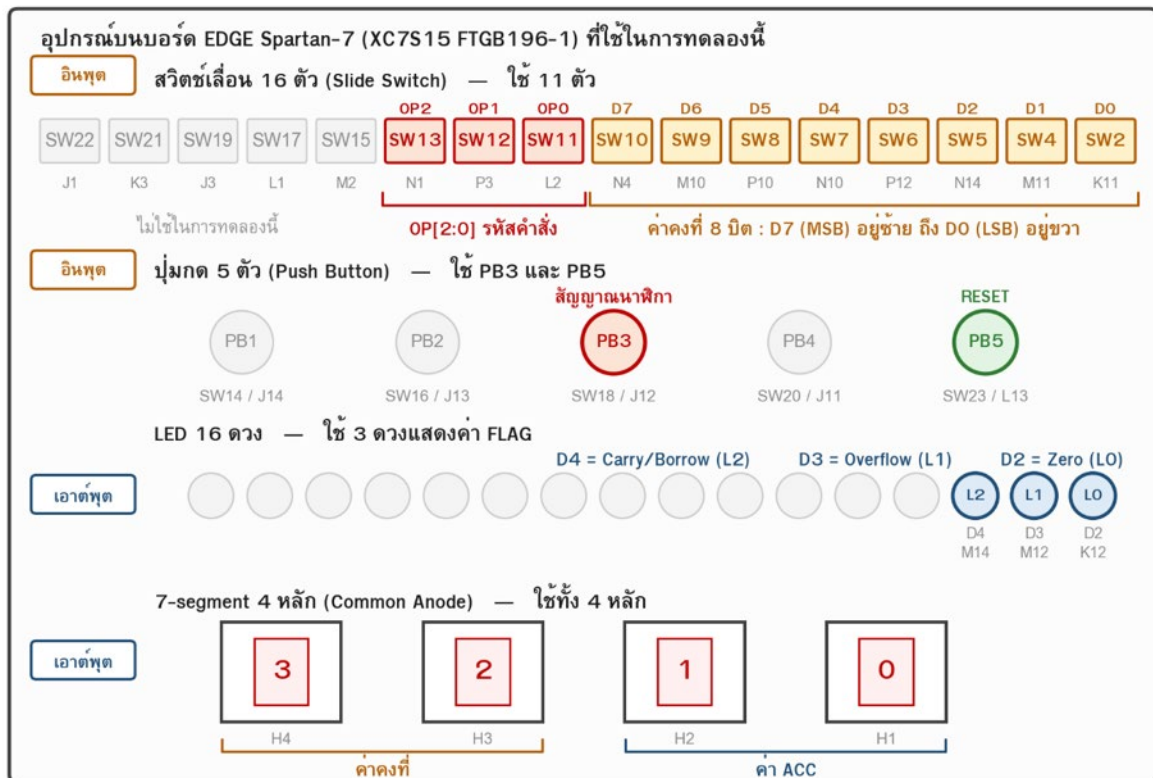
- 7.1. หน่วยความจำสำหรับเก็บคำสั่ง แทนตารางโปรแกรมบนกระดาษ
- 7.2. PC ที่เป็นวงจรรับ เพิ่มค่าเองทุกครั้งที่จบคำสั่ง และโหลดค่าใหม่ได้เมื่อมีคำสั่งกระโดด
- 7.3. IR ที่เป็นรีจิสเตอร์รับคำสั่งจากหน่วยความจำ แทนการอ่านค่าจากสวิตช์
- 7.4. CU ที่เป็นวงจรถอดรหัสคำสั่งร่วมกับวงจรสถานะจำกัด (Finite State Machine) เพื่อออกสัญญาณควบคุมตามลำดับ Fetch แล้ว Decode แล้ว Execute แทนมือของนักศึกษา
- 7.5. ใช้สัญญาณนาฬิกาจริง 50 MHz จากขา H11 แทนการกดปุ่ม

นั่นหมายความว่า สิ่งที่นักศึกษาทำด้วยมือในการทดลองนี้ทุกขั้นตอน คือสิ่งที่วงจรควบคุมของ CPU จริงทำแทนให้โดยอัตโนมัติ

บอร์ดและอุปกรณ์ที่ใช้ในการทดลอง

1. อุปกรณ์บนบอร์ดที่ใช้ในการทดลองนี้

การทดลองนี้ใช้บอร์ด EDGE Spartan-7 ซึ่งใช้ FPGA เบอร์ XC7S15 FTGB196-1 และมีตัวโปรแกรม USB JTAG อยู่บนบอร์ด อุปกรณ์บนบอร์ดที่ใช้ในการทดลองนี้ประกอบด้วย สวิตช์เลื่อน 16 ตัว ปุ่มกด 5 ตัว LED 16 ดวง 7-segment ชนิดแอโนดร่วม (Common Anode) จำนวน 4 หลัก และสัญญาณนาฬิกา 50 MHz



รูปที่ 5 อุปกรณ์บนบอร์ด EDGE Spartan-7 ที่ใช้ในการทดลองนี้

2. การกำหนดขาสัญญาณ

ตารางที่ 2 การกำหนดขา FPGA ของสัญญาณที่ใช้ในการทดลอง

ชื่อสัญญาณ	อุปกรณ์บนบอร์ด	ขา FPGA	ทิศทาง
CLK_50M	สัญญาณนาฬิกาบนบอร์ด 50 MHz	H11	เข้า
OP[2], OP[1], OP[0]	สวิตช์เลื่อน SW13, SW12, SW11	N1, P3, L2	เข้า
CONST[7] ถึง CONST[4]	สวิตช์เลื่อน SW10, SW9, SW8, SW7	N4, M10, P10, N10	เข้า
CONST[3] ถึง CONST[0]	สวิตช์เลื่อน SW6, SW5, SW4, SW2	P12, N14, M11, K11	เข้า
STEP	ปุ่มกด PB3 (SW18)	J12	เข้า
RESET	ปุ่มกด PB5 (SW23)	L13	เข้า
FLAG_CARRY (L2)	LED D4	M14	ออก

ชื่อสัญญาณ	อุปกรณ์บนบอร์ด	ขา FPGA	ทิศทาง
FLAG_OVERFLOW (L1)	LED D3	M12	ออก
FLAG_ZERO (L0)	LED D2	K12	ออก
SEG_A ถึง SEG_G และ DP	ส่วนแสดงผลของ 7-segment	L3 , P4 , P2 , M3 , M1 , J4 , K4 , J2	ออก
DIGIT1 ถึง DIGIT4	ตัวเลือกหลักของ 7-segment	H4 , H3 , H2 , H1	ออก

ลำดับในตารางที่ 2 และในรูปที่ 5 เรียงตามตำแหน่งจริงของอุปกรณ์บนบอร์ด เมื่อหันด้านที่อ่านตัวอักษรบนแผ่นวงจรได้ตามปกติ สวิตช์เลื่อนตัวซ้ายสุดคือ SW22 และตัวขวาสุดคือ SW2 ส่วนหลอด LED ที่ใช้ทั้งสามดวงอยู่ทางขวาของแถว จึงกำหนดให้บิตนัยสำคัญสูงสุดอยู่ซ้ายและบิตนัยสำคัญต่ำสุดอยู่ขวาทั้งสวิตช์และ LED การจับคู่ลำดับบิตนี้เป็นการกำหนดขึ้นเพื่อใช้ในการทดลอง

การทดลอง

1. ออกแบบวงจรจำลองการทำงานของ CPU ที่มีส่วนประกอบ 3 ส่วน คือ ALU ACC และ FLAG โดยโปรแกรมวาดวงจร schematic ที่จัดเตรียมไว้ให้ โดยวาดแต่ละบล็อกแยกกันตามผังวงจรที่ออกแบบ
 - 1.1. มี ACC ขนาด 8 บิต โดยเมื่อเปิดเครื่องหรือเมื่อกดปุ่ม RESET กำหนดให้ ACC เท่ากับ 0 และ FLAG ทุกบิตเท่ากับ 0
 - 1.2. มี ALU 1 ตัว ที่สามารถ บวก (+) ลบ (-) XOR (^) Logical Shift Left (<<) AND OR และ MOV โดยกำหนดให้ ACC เป็นตัวตั้งของการดำเนินการเสมอ
 - 1.3. Operation จะกระทำกับ ACC และเก็บผลลัพธ์ไว้ใน ACC เมื่อกดปุ่ม PB3 เปรียบเสมือน PB3 เป็น Clock

1.4. INPUT

- 1.4.1. สวิตช์เลื่อน SW13 ถึง SW11 เป็น OP[2:0] สำหรับ Operation 3 บิต โดย SW13 เป็นบิต OP[2] ซึ่งเป็นบิตนัยสำคัญสูงสุด (MSB) อยู่ซ้ายสุด และ SW11 เป็นบิต OP[0] ซึ่งเป็นบิตนัยสำคัญต่ำสุด (LSB) อยู่ขวาสุด

OP[2:0]	ความหมาย
000	ไม่มีการดำเนินการใด ๆ
001	+
010	-
011	^
100	<<
101	AND
110	OR
111	MOV (การกำหนดค่าของ ACC)

- 1.4.2. สวิตช์เลื่อน SW10 ถึง SW2 สำหรับค่าคงที่ 8 บิต โดย SW10 เป็นบิต D7 ซึ่งเป็น MSB อยู่ซ้ายสุด และ SW2 เป็นบิต D0 ซึ่งเป็น LSB อยู่ขวาสุด
- 1.4.3. ปุ่มกด PB3 สำหรับสั่งให้วงจรทำงาน 1 คำสั่ง
- 1.4.4. ปุ่มกด PB5 สำหรับ RESET

1.5. OUTPUT

- 1.5.1. 7-segment หลักที่ [1:0] แสดง ACC ที่ได้หลังจากทำ Operation เสมอ ในกรณี Overflow หรือ Carry ให้ตัดส่วนเกินทิ้ง
- 1.5.2. 7-segment หลักที่ [3:2] แสดงค่าคงที่จากสวิตช์เลื่อน SW10 ถึง SW2
- 1.5.3. หลอดไฟ LED แสดง FLAG ที่ได้หลังจาก ALU ทำ Operation โดยเรียงจากซ้ายไปขวาคือบอร์ดคือ D4 D3 และ D2
 - L2 คือ LED D4 สำหรับ Carry/Borrow Flag ผลลัพธ์มีการทศออกไป หรือยืมเข้ามา
 - L1 คือ LED D3 สำหรับ Overflow Flag ผลลัพธ์เกินจำนวนบิตที่รองรับได้
 - L0 คือ LED D2 สำหรับ Zero Flag ผลลัพธ์เป็น 0

2. รันโปรแกรมทีละคำสั่ง แล้วบันทึกผลการทดลอง
 - 2.1. ทดลองเดินโปรแกรมที่ 1 ตามตารางที่ 3 โดยเริ่มจากที่อยู่ 0 ตั้งสวิตช์ให้ตรงกับคำสั่งในแถวนี้ กดปุ่ม PB3 หนึ่งครั้ง อ่านค่าจาก 7-segment และ LED แล้วบันทึกลงตารางที่ 4 จากนั้นจึงเลื่อนไปยังที่อยู่ถัดไป ห้ามกดปุ่มซ้ำโดยไม่บันทึกผล
 - 2.2. เปรียบเทียบผลที่อ่านได้จากบอร์ดกับค่าที่คำนวณไว้ในตารางที่ 3 ถ้าไม่ตรงกันให้ระบุว่าคลาดเคลื่อนที่คำสั่งใด และวิเคราะห์ว่าเกิดจากส่วนใดของวงจร
 - 2.3. เขียนโปรแกรมของตนเองลงในตารางที่ 5 โดยมีเงื่อนไขว่า ค่าสุดท้ายของ ACC ต้องเท่ากับเลขสองหลักสุดท้ายของรหัสนักศึกษา ต้องมีครบทุก Operation ต้องต้องมีอย่างน้อยหนึ่งคำสั่งที่ทำให้ Zero Flag ติด (ไม่นับตอนเริ่มต้นก่อนเริ่มทำงาน) และอย่างน้อยหนึ่งคำสั่งที่ทำให้ Carry Flag ติด จากนั้นทดลองเดินโปรแกรมบนบอร์ดจริงเพื่อยืนยันผล

ตัวอย่าง โปรแกรมที่ 1 และการเดินทีละคำสั่ง

ตารางที่ 3 โปรแกรมตัวอย่างความยาว 8 คำสั่ง พร้อมผลลัพธ์ที่ควรได้ในแต่ละคำสั่ง (ค่า ACC เป็นเลขฐานสิบหก)

ที่อยู่ (PC)	คำสั่ง 11 บิต (OP ค่าคงที่)	ความหมายของคำสั่ง	ACC ก่อน	ACC หลัง	Zero L0	Overflow L1	Carry L2
0	100 0110 1001	$ACC \leftarrow ACC \ll 69$	00	00	1	0	0
1	101 1000 0000	$ACC \leftarrow ACC \text{ AND } 80$	00	00	1	0	0
2	000 0000 0000	ไม่มีการดำเนินการใด ๆ	00	00	1	0	0
3	111 1111 1111	$ACC \leftarrow FF$ (MOV)	00	FF	0	0	0
4	011 1101 0100	$ACC \leftarrow ACC \wedge D4$	FF	2B	0	0	0
5	101 1100 0110	$ACC \leftarrow ACC \text{ AND } C6$	2B	02	0	0	0
6	011 1000 0000	$ACC \leftarrow ACC \wedge 80$	02	82	0	0	0
7	111 0000 0101	$ACC \leftarrow 05$ (MOV)	82	05	0	0	0

ตัวอย่างการเดินคำสั่งที่อยู่ 4 ก่อนเริ่มคำสั่งนี้ ACC มีค่า FF ซึ่งเป็นผลจากคำสั่งที่อยู่ 3 นักศึกษาอ่านคำสั่งจากตาราง ได้รหัสปฏิบัติการ 011 คือ XOR และค่าคงที่ 1101 0100 คือ D4 จึงเลื่อนสวิตช์ SW13 SW12 SW11 ให้เป็น 0 1 1 และเลื่อนสวิตช์ SW10 ถึง SW2 ให้เป็น 1101 0100 เมื่อกดปุ่ม PB3 หนึ่งครั้ง ALU จะคำนวณ $FF \text{ XOR } D4$ ได้ 2B แล้ว ACC เก็บค่า 2B จอ 7-segment จะแสดง D4 ที่หลักที่ 3 และ 2 และแสดง 2B ที่หลักที่ 1 และ 0 ส่วนไฟ LED ทั้งสามดวงดับ เพราะผลลัพธ์ไม่เป็นศูนย์ ไม่มีการทด และไม่เกินขอบเขต

ตารางที่ 4 ตารางบันทึกผลที่อ่านได้จากบอร์ดจริง (โปรแกรมที่ 1)

ที่อยู่ (PC)	ค่าที่อ่านได้จาก 7-segment หลัก 3-2	ค่าที่อ่านได้จาก 7-segment หลัก 1-0	LED D4	LED D3	LED D2	ตรงกับตารางที่ 3 หรือไม่
0						
1						
2						
3						
4						
5						
6						
7						

ตารางที่ 5 โปรแกรมที่ 2 ซึ่งนักศึกษาเขียนขึ้นเอง

ที่อยู่ (PC)	คำสั่ง 11 บิต (OP ค่าคงที่)	ความหมายของคำสั่ง	ACC ก่อน	ACC หลัง	Zero L0	Overflow L1	Carry L2
0							
1							
2							
3							
4							
5							
6							
7							
8							
9							
10							
11							
12							
13							
14							
15							

คำถามท้ายการทดลอง

1. ถ้าลอจิสเตอร์ ACC ออก แล้วต่อเอาต์พุตของ ALU กลับไปเป็นอินพุตของ ALU โดยตรง วงจรจะทำงานอย่างไร และเพราะเหตุใด
2. ในการทดลองนี้ นักศึกษาทำหน้าที่แทนส่วนใดของ CPU บ้าง และถ้าต้องการให้วงจรเดินโปรแกรมได้เองโดยไม่ต้องกดปุ่ม ต้องเพิ่มวงจรอะไรบ้าง
3. Carry กับ Overflow ต่างกันอย่างไร จงยกตัวอย่างการบวกที่ทำให้ Carry ติดแต่ Overflow ไม่ติด และการบวกที่ทำให้ Overflow ติดแต่ Carry ไม่ติด
4. เพราะเหตุใดการกดปุ่มเพียงครั้งเดียวจึงอาจทำให้ ACC เปลี่ยนค่าหลายครั้ง และนักศึกษาแก้ปัญหานี้ด้วยวิธีใด
5. เมื่อมองจากมุมมองของ ALU คำสั่ง MOV ต่างจากคำสั่งอื่นอย่างไร และถ้าไม่มีคำสั่ง MOV จะยังกำหนดค่าเริ่มต้นให้ ACC ได้หรือไม่ ด้วยวิธีใด